Aegis backend deployment to Oracle Cloud Infrastructure

Step by step code release for an existing OCI OKE installation
Windows PowerShell | ags_sprint1 | 16 October 2026

Use this guide to publish a tested Aegis code revision to an already prepared OCI Kubernetes Engine installation. The backend contains the FastAPI API and durable worker. Existing Oracle Autonomous AI Database, Object Storage, shared application state and model settings must already be configured.

Release scope: API and worker are released together. Use **-Components Backend** to keep frontend unchanged; the default All releases both application components. The pilot uses Recreate deployments, so a release can briefly interrupt service. [1, 4]

Run this file: `ags_deploy/deploy-aegis-code.ps1`. It updates prepared workloads. Cluster creation, schema/wallet setup, Secrets and account initialization are separate tasks. [1]

Backend versions declared by the release source

Component	Declared version or build setting
Python base image	Python 3.12.15; immutable live-matching base digest
API server	FastAPI 0.142.2 and Uvicorn 0.54.0
Agent runtime	LangGraph 1.2.14
Oracle connectivity	python-oracledb 26.0.1; OCI SDK 2.187.2
CPU reranker	ONNX Runtime 1.30.0 and tokenizers 0.23.2

Follow the release in order

Steps	Task	Page
1 - 2	Prepare the checkout and run local checks	2
3 - 4	Select the destination and verify access	3
5 - 6	Prepare the release window and execute the script	4
7	Check workloads and one complete application workflow	5
8	Inspect a failed release and restore compatible images	6
9	Record the result and resolve common failures	7

Read-only live checks verified the schema, image bases and installed package versions. Source tests and local ARM64 image checks support the release. No images were published, database objects changed or workloads updated during this review.

1 Prepare the source

Use the existing Git checkout or clone the Aegis repository on branch **ags_sprint1**. Open PowerShell at its root, where ags_backend, ags_frontend and ags_deploy are present. Replace the example source path. Run commands individually and stop on any error.

```
$ErrorActionPreference = 'Stop'
Set-Location 'C:\Aegis'
git status --short --branch
if (git status --porcelain) { throw 'Save pending work before release.' }
git switch ags_sprint1
if ($LASTEXITCODE -ne 0) { throw 'Branch selection failed.' }
git pull --ff-only origin ags_sprint1
if ($LASTEXITCODE -ne 0) { throw 'Git update failed.' }
$taskSource = (Get-Location).Path
$taskCommit = git rev-parse HEAD
```

Keep any uncommitted work; resolve it before selecting the release revision. Record taskCommit with the release evidence. A branch name can move, so always retain the exact commit used for the build.

2 Check tools and test the release

Use a working Docker engine in Linux-container mode, Git, kubectl, Python 3.12 and Node.js 22. Docker must be able to build the selected target platform. The kubeconfig authentication helper and its required OCI identity must work on this computer. Standard OKE local access uses OCI CLI; an existing pilot kubeconfig may use the repository OCI SDK helper. [3]

```
git --version
docker version
kubectl version --client
```

On the first setup, create the virtual environment with the first line below. Skip that line when the existing .venv is ready. Install the test dependencies and run the isolated backend suite:

```
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install `
  -r ags_testing\backend\requirements.txt
.\.venv\Scripts\python.exe -B ags_testing\run_backend_tests.py
```

The backend runner uses temporary SQLite and mock providers, and clears inherited cloud settings and opt-in integration flags. Run frontend checks when frontend is selected, or when reviewing the complete application release:

```
npm --prefix ags_frontend ci
npm --prefix ags_frontend run test:unit
npm --prefix ags_frontend run typecheck
```

Expected result: no failed checks. The release script performs the production image builds later. The recorded local validation on 16 October 2026 was 1,047 backend tests passed, 5 skipped, 167 frontend tests passed, type checks passed and a Linux standalone production build passed. The 27 browser tests were listed, not executed. These results are a source snapshot, not a guarantee for later code changes.

3 Select the existing OCI destination

Obtain the authorized destination kubeconfig. A new Git clone does not contain private kubeconfigs or credentials. If it is missing, use the OKE cluster's **Access cluster > Local Access** instructions and the matching OCI identity. A private API endpoint also needs its approved network route. [3]

```
$taskKubeconfig = 'C:\aegis-target\kubeconfig.yaml'
kubectl --kubeconfig $taskKubeconfig config get-contexts
```

Copy the correct context name from that output, then replace every destination placeholder below. The registry namespace is the tenancy's **Object Storage namespace**; it is separate from the Kubernetes namespace. [2]

```
$taskContext = 'YOUR_CONFIRMED_CONTEXT'
$taskRegistry = 'bom.ocir.io'
$taskRegistryNamespace = 'YOUR_TENANCY_NAMESPACE'
$taskRegistryUser = 'YOUR_OCIR_LOGIN_USERNAME'
$taskNamespace = 'aegis-oci'
$taskPlatform = 'linux/arm64'
$taskRelease = 'ags-sprint1-' + (Get-Date -Format 'yyyyMMdd-HHmmss')
$taskKubeArgs = @('--kubeconfig', $taskKubeconfig,
  '--context', $taskContext, '-n', $taskNamespace)
```

bom.ocir.io is the Mumbai registry used by the reference installation; select the destination registry for another region. Choose linux/arm64 for A1/ARM nodes and linux/amd64 for an x86 destination. The existing pilot SDK helper at ags_deploy/ags_oke/ags_kubeconfig.py also requires its private provision state and OCI profile; it is not a general new-tenancy installer.

4 Verify access and registry login

```
kubectl --kubeconfig $taskKubeconfig --context $taskContext `
  config view --minify -o jsonpath='{.clusters[0].cluster.server}'
kubectl @taskKubeArgs get deployments aegis-web aegis-api-worker
kubectl @taskKubeArgs get pods,pvc,services
kubectl @taskKubeArgs auth can-i patch deployments
kubectl --kubeconfig $taskKubeconfig --context $taskContext `
  get nodes -o 'custom-columns=NAME:.metadata.name,ARCH:.status.nodeInfo.architecture'
docker login $taskRegistry --username $taskRegistryUser
```

For OCIR, use namespace/username for an IAM user, or the namespace/domain-name/username format for a federated user. Generate an auth token from the OCI profile's Tokens and keys / Auth Tokens section if needed, and enter that token at Docker's password prompt. Keep it out of command arguments. [2]

Expected result: the server/context are the intended target, nodes are Ready with the chosen architecture, patch access returns yes, both deployments exist and PVCs are Bound. Namespace aegis-oci must already contain web in aegis-web, and api plus worker in aegis-api-worker. The cluster also needs working private image-pull credentials for both repositories; Docker login on the laptop does not provide them. [1, 5]

5 Prepare the release window

Finish active chat, uploads, indexing, extraction and re-extraction before releasing. Confirm the destination already has its working Oracle connection/wallet, OCI content storage, shared /data state, stable encryption key, runtime configuration, CPU reranker files and model registrations. Retain compatible previous images. No backup is required for this code/object-only release.

The script changes container images. It does not run the database installer or recreate OCI resources. Confirm that the selected code remains compatible with existing schema and state. A Recreate update can stop the old Pods before new ones are ready. [1, 4]

6 Run the deployment script

This is the application-changing step. Review the source, target context, registry and platform values first. The prompted JSON route is ags_deploy/deploy.ps1 -Inputs C:\aegis-target\target.json (see README). It asks CONNECT and PUBLISH. The direct helper below publishes immediately after your separate approval. [1]

```
$taskDeploy = Join-Path $taskSource `
  'ags_deploy/deploy-aegis-code.ps1'
& $taskDeploy `
  -SourcePath $taskSource `
  -Kubeconfig $taskKubeconfig `
  -Context $taskContext `
  -Registry $taskRegistry `
  -RegistryNamespace $taskRegistryNamespace `
  -Namespace $taskNamespace `
  -Platform $taskPlatform `
  -Release $taskRelease `
  -Components Backend
```

Phase	What the current script does
Check and build	Checks workload/container names, builds selected images and checks their platform.
Publish	Pushes selected images before changing deployments; resolves repository digests.
Record	Saves the context/server, previous images and new digests in a private JSON receipt.
Roll out	Updates selected components only; API and worker share the backend digest. Each wait allows 15 minutes.

Backend builds enable Oracle/OCI and the existing AWS/Google native model SDKs. Their transitive dependencies are pinned to live versions. Local in-process embeddings stay disabled, matching live. The CPU reranker and existing Ollama/Gemini transports remain available; no model files or registrations are replaced. [1]

Expected finish message: Image rollouts completed. Keep the printed JSON receipt under ignored `ags_deploy/deployment-records/`. After an error, inspect actual workload images and the receipt before retrying. Partial rollouts are possible; rollback is manual. [1]

The direct helper has no -WhatIf or -CheckOnly mode. -SkipBuild requires the exact verified local release tags and still pushes and deploys them. For a later code change, select a new release value.

7 Verify the application release

Use the same destination variables. Verify readiness and inspect the actual images; successful image publication alone is not a completed release. [1, 4]

```
kubectl @taskKubeArgs rollout status `
  deployment/aegis-api-worker --timeout=15m
kubectl @taskKubeArgs rollout status `
  deployment/aegis-web --timeout=15m
kubectl @taskKubeArgs get deployments,pods,pvc

$taskObserved = kubectl @taskKubeArgs get deployments `
  aegis-web aegis-api-worker -o json | ConvertFrom-Json
$taskObserved.items | ForEach-Object {
  $_.metadata.name
  $_.spec.template.spec.containers | Select-Object name,image
}
```

Expected result: each deployment has its desired ready replicas, all relevant containers are ready and claims remain Bound. In the recorded pilot each deployment has one replica. For selected components, API and worker use backend_image and web uses web_image. Unselected images remain at their recorded previous digests. Inspect supporting containers without changing them.

```
$taskAppUrl = 'https://YOUR_EXISTING_AEGIS_HOST'
Invoke-WebRequest ($taskAppUrl.TrimEnd('/') + '/api/auth/status') `
  -UseBasicParsing | Select-Object StatusCode
```

Expected result: HTTPS succeeds and the public auth-status endpoint returns 200. Then use the normal browser UI with an existing authorized account; keep the current database, users and mappings.

Run one approved harmless workflow

1. Sign in and open the intended test dataset. Confirm that its model mappings and storage destination are the expected ones.

2. Upload a small fictional TXT/PDF using the normal workflow. Confirm the document, dataset and any external-processing approval. Wait for indexing to finish.

3. Run extraction with a valid saved prompt. Compare fields with the original, mark Reviewed and confirm the saved result remains after refresh.

4. Ask a question within the authorized dataset/document scope. Confirm scope/access checks, hybrid semantic and keyword retrieval, CPU reranking, generation, and answer/citation validation. Open the citation and compare it with the original evidence.

5. Record the outcome, deployed image references and any failed step with the receipt. A readiness result or connection test does not establish the complete workflow result.

These steps can create test records and invoke the configured model provider. Run them only in the approved dataset during the authorized release. No workflow was executed to prepare this PDF.

8 Inspect failures and restore images

After an error, stop and inspect the actual state. Backend may have updated while web remains old. Read recent API/worker logs and events; keep diagnostic output private if it contains workspace data. [1]

```
kubectl @taskKubeArgs get deployments,pods
kubectl @taskKubeArgs get events --sort-by=.metadata.creationTimestamp
kubectl @taskKubeArgs logs deployment/aegis-api-worker -c api --tail=80
kubectl @taskKubeArgs logs deployment/aegis-api-worker -c worker --tail=80
```

Choose the failed release's printed receipt path. Before restoring code, confirm its images are still available and compatible with current database/schema, configuration and shared state. The receipt is private operator evidence, not a database backup.

```
$taskReceiptPath = 'C:\path\to\the-failed-release-receipt.json'
$taskRecord = Get-Content -LiteralPath $taskReceiptPath -Raw |
  ConvertFrom-Json
$taskServer = kubectl --kubeconfig $taskKubeconfig `
  --context $taskContext config view --minify `
  -o jsonpath='{.clusters[0].cluster.server}'
if ($taskRecord.context -ne $taskContext -or
    $taskRecord.namespace -ne $taskNamespace -or
    $taskRecord.server -ne $taskServer) { throw 'Receipt target differs.' }
$taskOld = @{}
foreach ($taskDep in $taskRecord.previous) {
  foreach ($taskContainer in $taskDep.containers) {
    $taskOld["$($taskDep.deployment)/$($taskContainer.name)"] =
      $taskContainer.image
  }
}
$taskPreviousApi = $taskOld['aegis-api-worker/api']
$taskPreviousWorker = $taskOld['aegis-api-worker/worker']
$taskPreviousWeb = $taskOld['aegis-web/web']
if (-not $taskPreviousApi -or -not $taskPreviousWorker -or
    -not $taskPreviousWeb) { throw 'Previous image references missing.' }
```

Run the backend block only when its images changed and restoring them is appropriate:

```
kubectl @taskKubeArgs set image deployment/aegis-api-worker `
  "api=$taskPreviousApi" "worker=$taskPreviousWorker"
kubectl @taskKubeArgs rollout status `
  deployment/aegis-api-worker --timeout=15m
```

Run the web block only when web changed and needs restoration:

```
kubectl @taskKubeArgs set image deployment/aegis-web "web=$taskPreviousWeb"
kubectl @taskKubeArgs rollout status deployment/aegis-web --timeout=15m
```

Code rollback does not restore database rows, schema, files or credentials. If those changed, use a reviewed compatible recovery plan. Recheck readiness and one approved workflow after restoring images. [1, 4]

9 Record the outcome and resolve errors

Retain the exact source commit, release tag, destination context/server/namespace, platform, receipt path, previous/new image digests, operator/date and actual workflow outcome. Use the private release evidence location; do not commit credentials or receipt contents. An unsuccessful release remains incomplete until actual workload state is understood.

Symptom	What to inspect before retrying
Context, timeout or Forbidden	Kubeconfig helper, OCI identity, network route, namespace and Kubernetes permissions.
Docker login or push denied	Destination registry, tenancy namespace, auth token, username and repository push permission. [2]
ImagePullBackOff	Published digest and the workload's private image-pull credentials in its namespace. [5]
Wrong executable format	Selected image platform versus the destination worker architecture.
Init or readiness failure	Prepared /data state, wallet, stable encryption key, database access, runtime profile and reranker files.
Rollout timeout or mixed versions	Deployment images, Pod events, API/worker logs and that release's receipt. No automatic rollback runs. [1]
Model or workflow failure	Registered endpoint/credentials, limits, enabled roles, dataset mappings and available reranker.

Source references

Repository commands and supported parameters were verified against the source below. Official platform references were checked on 16 October 2026. Bracketed numbers identify their use in this guide.

[1] Aegis deployment script at a7a02dd

[2] Oracle Container Registry push and login

[3] Oracle OKE cluster access

[4] Kubernetes deployment updates and rollback

[5] Kubernetes private image pulls

Additional repository evidence: ags_backend/Dockerfile, its runtime requirements, ags_testing/README.md and the maintained architecture/application deployment guides. The separate database deployment PDF covers fresh Oracle schema installation.

Source commit: a7a02dd3e136253b2785bb69025d389fb293e412

Deployment script SHA256

```
a0c08d502c8cc5fc3fe5e5cf28ab4c4e679b481f48a39ed14d98dd869bdccb3e
```

Preparation boundary: PowerShell samples were syntax checked, and every PDF page was rendered and visually reviewed. Live inspection was read-only; no OCI resource, database object/data, registry image, Kubernetes workload or application setting was changed.